Assignment 2 – Experimental Robotics

PlanSys2 mission: Explore waypoints and take pictures of ArUco markers

Package: **my_opencv**

Date: 2026-01-06

Josselin Gressard Léo De Sue Mathis Giau Thomas Pradinat Victor Rossignol

Repository: *https://github.com/josselin06/Assignement2_Experimental_Robotics*

This document explains the behavior and implementation of the PlanSys2 actions used in Assignment 2: explore and take_picture_next.

Table of contents

Table of contents 2

1. Context and objective 3

2. System overview 3

2.1 Component diagram (high level) 3

2.2 ROS interfaces and persistence 3

3. Action “explore” (ExploreAction) 5

3.1 Inputs, parameters and waypoint list 5

3.2 Internal state machine 5

3.3 Map clipping and approach search 5

3.4 ArUco detection and stored outputs 6

4. Action“take_picture_next” (TakePictureNextAction) 7

4.1 Selecting the next marker to process 7

4.2 Navigation to the marker area 7

4.3 Search rotation and centering control 7

4.4 Publishing the “picture” and marking the ID as done 7

5. Planning model (PDDL) 8

5.1 Domain description (high-level) 8

5.2 Problem instance and goal 8

5.3 Example plan 8

6. Launch file and execution procedure 8

6.1 What the launch file does 8

6.2 How to run (typical sequence) 9

1. Context and objective

The goal of this assignment is to build an autonomous mission controlled by PlanSys2, where a robot uses a camera to detect ArUco markers while navigating with Nav2. The mission is split into two main phases:

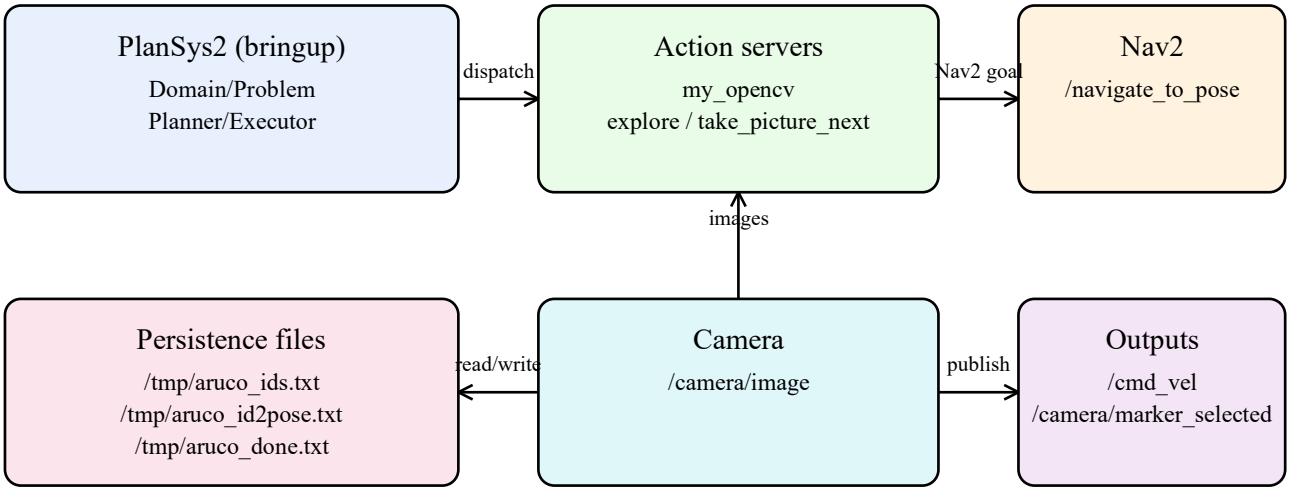
- Exploration: the robot visits a fixed set of waypoints (wp1…wp4). At each waypoint, it observes the environment for a short scan window and stores the detected marker IDs.
- Picture-taking: after exploration, the robot processes the detected markers one by one. For each marker, it navigates back to the associated area and rotates to center the marker in the camera image, then publishes an annotated image.

These behaviors are implemented as two PlanSys2 action servers in the package my_opencv: explore_action (PDDL action *explore*) and take_picture_next_action (PDDL action *take_picture_next*).

2. System overview

At runtime, PlanSys2 creates a plan from the PDDL domain/problem and dispatches actions to the corresponding action servers. The action servers interact with Nav2 (NavigateToPose), the camera topic, and a small set of persistent files in */tmp*.

2.1 Component diagram (high level)



2.2 ROS interfaces and persistence

Main ROS interfaces used by the two actions:

- Action client: /navigate_to_pose (Nav2) — used to move the robot to waypoints / marker areas.

- Subscription: /camera/image — raw camera frames for ArUco detection.
- Publisher (take_picture_next): /cmd_vel — angular velocity commands during the centering/search behavior.
- Publisher (take_picture_next): /camera/marker_selected — annotated image of the centered marker.
- Subscription (explore): /map (OccupancyGrid, transient local) — used to keep intermediate “approach” goals inside the map and avoid occupied cells.

Persistent files (used to share information between actions):

- /tmp/aruco_ids.txt: set of all detected marker IDs (one ID per line).
- /tmp/aruco_id2pose.txt: mapping *ID → (x,y)* (one line: `id x y`). In this implementation, (x,y) is the waypoint position where the ID was first detected (approximate location).
- /tmp/aruco_done.txt: set of processed IDs (one ID per line), written by *take_picture_next*.

3. Action “explore” (ExploreAction)

The explore action is responsible for visiting a waypoint and collecting ArUco IDs. It is implemented as a class ExploreAction that inherits from plansys2::ActionExecutorClient (action server side in PlanSys2).

3.1 Inputs, parameters and waypoint list

Waypoints are hard-coded in the constructor (wp1…wp4) with positions in the map frame.

Important parameters (selected):

- image_topic, map_topic: input topics.
- nav_action_name, nav_goal_frame: Nav2 action name and goal frame (usually map).
- scan_duration_s: duration of the scan window after reaching the waypoint.
- detect_only_near_during_nav + detect_near_dist_m: allows marker detection during navigation only when the robot is close to the target waypoint (reduces false early detections).
- cancel_nav_on_new_marker: if a new marker is detected while approaching (near), Nav2 is canceled and the scan window starts immediately.
- enable_approach_search, approach_radius_m, approach_max_poses: if 0 marker is found at the waypoint, the robot tries several additional poses around it.

3.2 Internal state machine

The action is implemented as a finite state machine evaluated periodically in do_work(). Main phases (simplified):

- **WAIT_NAV_SERVER**: wait for Nav2 action server availability.
- **WAIT_MAP**: wait for a valid /map to enable map clipping and approach pose filtering.
- **SEND_NAV_GOAL** → **WAIT_NAV_RESULT**: send a NavigateToPose goal to the waypoint (possibly clipped inside map bounds).
- **WAIT_SCAN**: after reaching the waypoint (or after cancel), keep scanning images for scan_duration_s seconds.
- **DECIDE_AFTER_SCAN**: store results and finish the action; if 0 marker was found, optionally start approach search (SEND_APPROACH_GOAL / WAIT_APPROACH_NAV / WAIT_APPROACH_SCAN).

3.3 Map clipping and approach search

To avoid sending Nav2 goals outside the occupancy grid, the waypoint goal can be clipped inside the map bounds with a configurable margin (clip_margin_m). If a waypoint is outside the map, the node sends an intermediate clipped goal and retries (up to max_clip_steps).

If the scan window finds 0 marker and enable_approach_search is enabled, the node generates a set of poses on a circle around the waypoint and tries them sequentially. Each candidate pose is:

- Clipped inside the map bounds if needed.
- Rejected if the corresponding occupancy grid cell is occupied above approach_min_free_cell (unknown cells are accepted).

3.4 ArUco detection and stored outputs

The subscription callback image_cb() runs OpenCV ArUco detection (same parameters as my Assignment 1). When IDs are detected, the node updates:

- **ids_in_this_wp_**: IDs detected during the current waypoint processing.
- **global_found_ids_**: union of all IDs seen since the beginning (persisted to /tmp/aruco_ids.txt).
- **id_to_pose_**: mapping of each newly seen ID to the current waypoint position (persisted to /tmp/aruco_id2pose.txt).

A key design choice here is that id_to_pose stores the waypoint location where the marker was first seen, not the true 3D pose of the marker. This is sufficient to guide the robot back to the correct area in the second phase.

4. Action“take_picture_next” (TakePictureNextAction)

The take_picture_next action processes detected markers one by one. At each call, it selects the next ID not yet processed, navigates to the stored (x,y) location, then rotates in place until the marker is centered in the camera image. Finally, it publishes an annotated image on /camera/marker_selected and records the ID as done.

4.1 Selecting the next marker to process

The node reads two files:

- /tmp/aruco_ids.txt → *all detected IDs*
- /tmp/aruco_done.txt → *already processed IDs*

The helper function pick_next_id(all, done) selects the smallest ID that is in *all* but not in *done*. If no ID remains, the action returns success immediately (the planner may still call it again to advance its counters).

4.2 Navigation to the marker area

The node loads /tmp/aruco_id2pose.txt to retrieve the target position (x,y) for the selected ID and sends a Nav2 NavigateToPose goal. If the mapping is missing, the action fails with a clear error message.

4.3 Search rotation and centering control

After Nav2 succeeds, the node enters the CENTERING phase. The camera callback keeps an updated dictionary last_centers_ that associates each detected ID with its image center (pixel coordinates).

If the target marker is not currently visible, the robot rotates in place at a constant angular speed search_ang_vel. This implements the requested behavior “make a full turn on itself until the marker is seen”.

Once visible, the robot performs a proportional centering control on the horizontal image error. When |error_x| ≤ center_tolerance_px, the marker is considered centered and the robot stops.

4.4 Publishing the “picture” and marking the ID as done

When the marker is centered, the node clones the last camera frame and draws:

- All detected marker borders (cv::aruco::drawDetectedMarkers).
- A red circle around the target marker center.
- A text label with the target ID.

The annotated image is published on /camera/marker_selected. The ID is then appended to /tmp/aruco_done.txt, and the action returns success.

5. Planning model (PDDL)

PlanSys2 relies on a PDDL domain and problem. In this mission, PDDL is used to enforce the order:

- Explore all waypoints first.
- Then execute *take_picture_next* several times (counter-based) until the goal is reached.

5.1 Domain description (high-level)

The domain defines two actions:

- **explore(r, w, c1, c2)**: can be executed only if waypoint *w* is still *unexplored* and the exploration counter is at *c1*. It marks *w* explored and increments the exploration counter to *c2*.
- **take_picture_next(r, c1, c2, cmax)**: can be executed only when exploration is finished (the exploration counter equals the maximum *cmax*) and the picture counter is at *c1*. It increments the picture counter to *c2*.
- Counters (c0…c4) are linked by the predicate next and provide a simple way to bound the number of times actions are executed.

5.2 Problem instance and goal

The problem defines one robot (robot1), four waypoints (wp1…wp4), and counters (c0…c4). The goal requires:

- All waypoints are explored: (explored wp1)…(explored wp4)
- The picture counter reaches c4: (pc c4)

5.3 Example plan

A typical plan produced by the planner (one possible ordering) is:

```
explore(robot1, wp1, c0, c1)
explore(robot1, wp2, c1, c2)
explore(robot1, wp3, c2, c3)
explore(robot1, wp4, c3, c4)
take_picture_next(robot1, c0, c1, c4)
take_picture_next(robot1, c1, c2, c4)
take_picture_next(robot1, c2, c3, c4)
take_picture_next(robot1, c3, c4, c4)
```

In practice, *take_picture_next* selects the next unprocessed marker ID from /tmp/aruco_ids.txt. If fewer than four markers exist, the action still returns success (“all marker IDs already processed”), allowing the plan to complete and satisfy the counter-based goal.

6. Launch file and execution procedure

A dedicated launch file starts the PlanSys2 bringup (distributed) and the two action servers with consistent parameters (topics, timeouts, use_sim_time). The launch file also loads the domain and problem PDDL files from the package share.

6.1 What the launch file does

- Includes plansys2_bringup_launch_distributed.py with domain.pddl and problem.pddl.

- Starts explore_action node with parameters (scan duration, nav action name, map topic, reset files…).
- Starts take_picture_next_action node with parameters (cmd_vel topic, centering gains, reset done file…).

6.2 How to run (typical sequence)

The complete sequence to launch my code is explained on the Github README

Note: We sped up the video by 3× because otherwise it was too long and too large to upload. Also, please keep in mind that we couldn't drive the robot faster during the simulation, because our computer would crash/overheat (CPU overload), causing many LiDAR readings to be dropped. We even had to reduce the Nav2 controller frequency, otherwise the simulation would crash. We therefore did our best to obtain a stable and representative simulation with the hardware and tools available to us.